

【pytorch05】PyTorch 实现线性回归

2026年9月8日 8:02

• PyTorch 编程四步法:

1. **准备数据集** (Prepare dataset) ;
Eg.[1,2][2,4][3,6]
2. **用类设计模型** (Design model using Class, 继承 nn.Module) ;
计算y_hat
3. **构造损失函数和优化器** (Construct loss and optimizer, 用 PyTorch API) ;
4. **训练循环** (Training cycle: forward → backward → update) 。

$$\begin{bmatrix} y_{pred}^{(1)} \\ y_{pred}^{(2)} \\ y_{pred}^{(3)} \end{bmatrix} = \omega \cdot \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ x^{(3)} \end{bmatrix} + b$$

- x和y必须是矩阵, w是广播机制
- $y[3 \times 1] = w[3 \times 1](\text{广播}) \times x[3 \times 1]$

代码解释:

1. **nn.Module**: 所有神经网络模块的基类; 自定义模型需实现 `__init__()` 和 `forward()`。
2. **nn.Linear(in_features, out_features)**: 线性层 (仿射变换 $y = Wx + b$) , 内部含 **weight** 和 **bias** 两个张量。
3. **__call__ 魔法方法**: nn.Module 实现了 `__call__`, 使实例可像函数一样调用, 且内部会自动调用 `forward()`。
4. **损失函数**: `torch.nn.MSELoss(size_average=...)`。
5. **优化器**: `torch.optim.SGD(model.parameters(), lr=0.01)`; `model.parameters()` 收集所有可学习参数。
6. **标准训练循环三步**:
 1. `optimizer.zero_grad()`: **清空梯度** (否则累加) ;
 2. `loss.backward()`: **反向传播求梯度**;
 3. `optimizer.step()`: **用优化器更新参数**。
7. **测试模型**: 输出 `model.linear.weight / model.linear.bias`; 对新输入直接 `model(x)` 得到预测。
8. **其他优化器 (练习)** : Adagrad、Adam、Adamax、ASGD、LBFGS、RMSprop、Rprop 等。

```
import torch
```

```
# 1. 准备数据集
```

```
x_data = torch.Tensor([[1.0], [2.0], [3.0]])
```

```
y_data = torch.Tensor([[2.0], [4.0], [6.0]])
```

```
# 2. 用类设计模型 (继承 nn.Module)
```

```
class LinearModel(torch.nn.Module):
```

```
    def __init__(self): #初始化函数
```

```

super(LinearModel, self).__init__() # 【必须写】!
self.linear = torch.nn.Linear(1, 1) # 输入维度1, 输出维度1
# 线性层, 自带 weight 和 bias

def forward(self, x):
    y_pred = self.linear(x) # 执行矩阵乘法:  $y_{pred} = x @ W.T + b$ 
    return y_pred

model = LinearModel() # 实例化

# 3. 构造损失函数和优化器
criterion = torch.nn.MSELoss(size_average=False) # 不求平均值, 影响不大
optimizer = torch.optim.SGD(model.parameters(), lr=0.01) # model.parameters() 会取出模型里所有的矩阵参数 (W 和 b)
# lr=0.01 是步长(learning rate)

# 4. 训练循环
for epoch in range(1000):
    y_pred = model(x_data) # Forward: 前向预测
    loss = criterion(y_pred, y_data) # 计算损失
    print(epoch, loss.item()) # .item()改为标量

    optimizer.zero_grad() # 清空梯度!!! (否则累加)
    loss.backward() # Backward: 反向传播
    optimizer.step() # Update: 更新参数

# 输出权重与偏置
print('w = ', model.linear.weight.item())
print('b = ', model.linear.bias.item())

# 测试模型
x_test = torch.Tensor([[4.0]])
y_test = model(x_test)
print('y_pred = ', y_test.data)

```